

Healthcare Patient Management System

Overview

Healthcare organizations manage patient, admission, treatment, laboratory, medication, and hospital outcome data. Staff need reliable systems to search records, monitor hospital activity, analyze patient outcomes, and prepare operational reports.

Manual handling of large healthcare datasets can make record management and reporting slow and inconsistent.

Healthcare teams often ask questions such as:

- How many admissions and readmissions occur across different hospitals or patient groups?
- What are the trends in length of stay and treatment activity?
- Which patient and encounter attributes are associated with readmission?
- How can staff quickly retrieve and update authorized patient records?

Build a Python-based full-stack healthcare management application that loads historical healthcare data, stores curated information in a relational database, provides CRUD and REST APIs, and presents operational dashboards.

Questions

- What are the admission and readmission trends?
- How does length of stay vary across patient groups and encounter types?
- Which hospitals or encounter categories have higher readmission rates?
- How can authorized users search, update, and report patient records efficiently?

Requirement 1 – Basic

- Build a Python ingestion pipeline for healthcare CSV files
- Clean categorical, numerical, missing, and date-related fields using Pandas
- Design normalized relational tables for patients, encounters, medications, and outcomes
- Load curated data into PostgreSQL/MySQL
- Implement CRUD operations for authorized patient and encounter records
- Build REST APIs using Flask or FastAPI
- Create dashboards for admissions, readmissions, length of stay, and encounter KPIs
- Implement validation, exception handling, and audit-friendly logging

Requirement 2 – Advanced

- Implement incremental loading of new encounter records
- Create data-quality rules for duplicates, invalid codes, missing values, and inconsistent dates
- Implement API filtering, pagination, sorting, and search
- Add role-based access for administrator, clinical operations, and analyst users
- Implement database indexes and query optimization for frequently used reports
- Create automated ETL and API tests

- Implement audit logs for create, update, and delete operations
- Add scheduled reporting and export to CSV/Excel

Build a simple front-end interface.

Deliverables

1. Architecture Diagram

- Include a high-level view of the system in JPEG or PDF.
- Show how data is processed from the provided dataset through ETL, database, APIs, and the user's interface.

2. Design

- Articulate system design decisions taken and the trade-offs made.

3. Full Executable Code (Microservice)

- Focus on clarity and modularity.
- Provide a README explaining:
 - Project setup (how to install or run locally)
 - Sample usage (for example, a test API request and resulting output)
 - Database setup and schema
 - ETL execution steps

4. Panel Presentation (10 minutes)

- Demo of the working solution outlining the design and approach (8 minutes).
- Q&A (2 minutes).

Dataset Name

Diabetes 130-US Hospitals for Years 1999-2008

Primary Link

UCI Machine Learning Repository – Diabetes 130-US Hospitals

<https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008>

Alternative Link

UCI Dataset Documentation / Download

<https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008>

Format

CSV

Key Fields

encounter_id, patient_nbr, race, gender, age, admission_type_id, time_in_hospital, num_lab_procedures, num_medications, number_outpatient, number_inpatient, number_emergency, diagnosis, insulin, readmitted

Reason

The dataset contains more than 100,000 hospital encounters and includes demographic, admission, treatment, medication, utilization, length-of-stay, and readmission information. It supports ETL, relational modeling, CRUD/search workflows, API development, and operational analytics. The UCI page notes that the data contains sensitive demographic attributes, so the project should use it only for educational analysis and avoid exposing unnecessary personal attributes.